QUESTION 1: Implement a Room Management System

Task Description: Extend the hotel management system to include room management functionality. Create a Room class hierarchy with different room types (Standard, Deluxe, Suite) and implement room booking/availability features.

Implementation:

Step 1: Create Abstract Room Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/Room.java`

```
package com.mycompany.hotelmanagementsystem;

public abstract class Room {
    private int roomNumber;
    private double basePrice;
    private boolean isAvailable;
    private String description;
    private int capacity;

    public Room(int roomNumber, double basePrice, int capacity) {
        this.roomNumber = roomNumber;
        this.basePrice = basePrice;
        this.capacity = capacity;
        this.isAvailable = true;
        this.description = "";
    }
```

```
    // Abstract methods to be implemented by subclasses
    public abstract double calculateTotalPrice(int nights);
    public abstract String getRoomType();

// Getters and Setters
    public int getRoomNumber() { return roomNumber; }
    public void setRoomNumber(int roomNumber) { this.roomNumber = roomNumber; }

    public double getBasePrice() { return basePrice; }
    public void setBasePrice(double basePrice) { this.basePrice = basePrice; }

    public boolean isAvailable() { return isAvailable; }
    public void setAvailable(boolean available) { isAvailable = available; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public int getCapacity() { return capacity; }
    public void setCapacity(int capacity) { this.capacity = capacity; }

    @Override
    public String toString() {
        return String.format("Room %d (%s) - $%.2f/night - %s - Capacity: %d",
            roomNumber, getRoomType(), basePrice,
            isAvailable ? "Available" : "Occupied", capacity);
    }}
```

Step 2: Create StandardRoom Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/StandardRoom.java`

```
package com.mycompany.hotelmanagementsystem;

public class StandardRoom extends Room {
  private boolean hasWiFi;

  public StandardRoom(int roomNumber, double basePrice) {
    super(roomNumber, basePrice, 2); // Standard capacity of 2
    this.hasWiFi = true;
    setDescription("Standard room with basic amenities");
  }

  @Override
  public double calculateTotalPrice(int nights) {
    return getBasePrice() * nights;
  }

  @Override
  public String getRoomType() {
    return "Standard";
  }

  public boolean hasWiFi() { return hasWiFi; }
  public void setHasWiFi(boolean hasWiFi) { this.hasWiFi = hasWiFi; }
```

```
    @Override
    public String toString() {
        return super.toString() + " - WiFi: " + (hasWiFi ? "Yes" : "No");
    }
}
```

Step 3: Create DeluxeRoom Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/DeluxeRoom.java`

```
package com.mycompany.hotelmanagementsystem;

public class DeluxeRoom extends Room {
    private boolean hasBalcony;
    private boolean hasMiniBar;
    private static final double LUXURY_MULTIPLIER = 1.5;

    public DeluxeRoom(int roomNumber, double basePrice) {
        super(roomNumber, basePrice, 3); // Deluxe capacity of 3
        this.hasBalcony = true;
        this.hasMiniBar = true;
        setDescription("Deluxe room with premium amenities");
    }
```

```
    @Override
    public double calculateTotalPrice(int nights) {
        return getBasePrice() * nights * LUXURY_MULTIPLIER;
    }

    @Override
    public String getRoomType() {
        return "Deluxe";
    }

    public boolean hasBalcony() { return hasBalcony; }
    public void setHasBalcony(boolean hasBalcony) { this.hasBalcony = hasBalcony; }

    public boolean hasMiniBar() { return hasMiniBar; }
    public void setHasMiniBar(boolean hasMiniBar) { this.hasMiniBar = hasMiniBar; }

    @Override
    public String toString() {
        return super.toString() + String.format(" - Balcony: %s, MiniBar: %s",
            hasBalcony ? "Yes" : "No", hasMiniBar ? "Yes" : "No");
    }
}
```

Step 4: Create Suite Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/Suite.java`

```
package com.mycompany.hotelmanagementsystem;

public class Suite extends Room {
  private int numberOfRooms;
  private boolean hasKitchen;
  private boolean hasLivingRoom;
  private static final double SUITE_MULTIPLIER = 2.5;

  public Suite(int roomNumber, double basePrice, int numberOfRooms) {
    super(roomNumber, basePrice, 6); // Suite capacity of 6
    this.numberOfRooms = numberOfRooms;
    this.hasKitchen = true;
    this.hasLivingRoom = true;
    setDescription("Luxury suite with multiple rooms");
  }

  @Override
  public double calculateTotalPrice(int nights) {
    return getBasePrice() * nights * SUITE_MULTIPLIER * numberOfRooms;
  }

  @Override
  public String getRoomType() {
```

```
        return "Suite";
    }

    public int getNumberOfRooms() { return numberOfRooms; }
    public void setNumberOfRooms(int numberOfRooms) { this.numberOfRooms = numberOfRooms; }

    public boolean hasKitchen() { return hasKitchen; }
    public void setHasKitchen(boolean hasKitchen) { this.hasKitchen = hasKitchen; }

    public boolean hasLivingRoom() { return hasLivingRoom; }
    public void setHasLivingRoom(boolean hasLivingRoom) { this.hasLivingRoom = hasLivingRoom; }

    @Override
    public String toString() {
        return super.toString() + String.format(" - Rooms: %d, Kitchen: %s, Living Room: %s",
            numberOfRooms, hasKitchen ? "Yes" : "No", hasLivingRoom ? "Yes" : "No");
    }
}
```

Step 5: Modify WestminsterHotelManager Class

File Location: Modify existing `WestminsterHotelManager.java`

Add these fields to the class:

```
private ArrayList<Room> hotelRoomList;
```

Add in constructor:

```
public WestminsterHotelManager(int maxMembersNumber) {
   hotelStaffList = new ArrayList<>();
   hotelGuestList = new ArrayList<>();
   hotelRoomList = new ArrayList<>(); // Add this line
   staffLimit = maxMembersNumber;
   initializeRooms(); // Add this line
}
```

Add these methods:

```
private void initializeRooms() {
   // Initialize with some default rooms
   for (int i = 101; i <= 110; i++) {
      hotelRoomList.add(new StandardRoom(i, 100.0));
   }
   for (int i = 201; i <= 205; i++) {
      hotelRoomList.add(new DeluxeRoom(i, 150.0));
   }
   for (int i = 301; i <= 303; i++) {
      hotelRoomList.add(new Suite(i, 300.0, 3));
   }
}

public void addRoom() {
   Scanner s = new Scanner(System.in);
```

```
System.out.println("Enter room number:");
int roomNumber = s.nextInt();

System.out.println("Enter base price:");
double basePrice = s.nextDouble();

System.out.println("Select room type:");
System.out.println("1. Standard Room");
System.out.println("2. Deluxe Room");
System.out.println("3. Suite");

int roomType = s.nextInt();
Room room = null;

switch (roomType) {
  case 1:
    room = new StandardRoom(roomNumber, basePrice);
    break;
  case 2:
    room = new DeluxeRoom(roomNumber, basePrice);
    break;
  case 3:
    System.out.println("Enter number of rooms in suite:");
    int numRooms = s.nextInt();
    room = new Suite(roomNumber, basePrice, numRooms);
    break;
```

```
        default:
            System.out.println("Invalid room type");
            return;
    }

    hotelRoomList.add(room);
    System.out.println("Room added successfully!");
}

public void printAvailableRooms() {
    System.out.println("\n=== AVAILABLE ROOMS ===");
    boolean hasAvailable = false;

    for (Room room : hotelRoomList) {
        if (room.isAvailable()) {
            System.out.println(room.toString());
            hasAvailable = true;
        }
    }

    if (!hasAvailable) {
        System.out.println("No rooms available");
    }
}

public void bookRoom() {
```

```
Scanner s = new Scanner(System.in);

printAvailableRooms();

System.out.println("Enter room number to book:");
int roomNumber = s.nextInt();

System.out.println("Enter number of nights:");
int nights = s.nextInt();

Room roomToBook = null;
for (Room room : hotelRoomList) {
   if (room.getRoomNumber() == roomNumber && room.isAvailable()) {
      roomToBook = room;
      break;
   }
}

if (roomToBook != null) {
   roomToBook.setAvailable(false);
   double totalPrice = roomToBook.calculateTotalPrice(nights);
   System.out.printf("Room %d booked successfully!%n", roomNumber);
   System.out.printf("Total price for %d nights: $%.2f%n", nights, totalPrice);
} else {
   System.out.println("Room not available or doesn't exist");
}}
```

QUESTION 2: Implement File I/O for Data Persistence

Task Description:

Add functionality to save and load hotel data (staff, guests, rooms) to/from files using both text files and serialization.

Implementation:

Step 1: Create HotelDataManager Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/HotelDataManager.java`

```
package com.mycompany.hotelmanagementsystem;

import java.io.*;

import java.time.LocalDate;

import java.time.format.DateTimeFormatter;

import java.util.ArrayList;

import java.util.List;


public class HotelDataManager {

   private static final String STAFF_FILE = "hotel_staff.txt";

   private static final String GUEST_FILE = "hotel_guests.txt";

   private static final String ROOM_FILE = "hotel_rooms.txt";

   private static final String BINARY_FILE = "hotel_data.dat";
```

```
// Save staff to text file
  public static void saveStaffToTextFile(List<HotelStaff> staffList) {
    try (PrintWriter writer = new PrintWriter(new FileWriter(STAFF_FILE))) {
      writer.println("# Hotel Staff Data");
      writer.println("# Format: Type|Name|Surname|StaffID|DateOfBirth|SpecificInfo");

      for (HotelStaff staff : staffList) {
        StringBuilder line = new StringBuilder();

        if (staff instanceof Manager) {
          Manager manager = (Manager) staff;
          line.append("Manager|")
            .append(staff.getName()).append("|")
            .append(staff.getSurname()).append("|")
            .append(staff.getStaffID()).append("|")
            .append(staff.getStringDate()).append("|")
            .append(manager.getLicenseNumber());
        } else if (staff instanceof HouseKeeper) {
          HouseKeeper housekeeper = (HouseKeeper) staff;
          line.append("HouseKeeper|")
            .append(staff.getName()).append("|")
            .append(staff.getSurname()).append("|")
            .append(staff.getStaffID()).append("|")
            .append(staff.getStringDate()).append("|")
            .append(housekeeper.getYearsOfExperience());
        }
```

```
            writer.println(line.toString());
        }

        System.out.println("Staff data saved to " + STAFF_FILE);

    } catch (IOException e) {
        System.err.println("Error saving staff data: " + e.getMessage());
    }
}

// Load staff from text file
public static List<HotelStaff> loadStaffFromTextFile() {
    List<HotelStaff> staffList = new ArrayList<>();
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");

    try (BufferedReader reader = new BufferedReader(new FileReader(STAFF_FILE))) {
        String line;

        while ((line = reader.readLine()) != null) {
            if (line.startsWith("#") || line.trim().isEmpty()) {
                continue; // Skip comments and empty lines
            }

            String[] parts = line.split("\\|");
            if (parts.length >= 6) {
```

```
        String type = parts[0];
        String name = parts[1];
        String surname = parts[2];
        String staffID = parts[3];
        LocalDate dateOfBirth = LocalDate.parse(parts[4], formatter);
        String specificInfo = parts[5];

        HotelStaff staff = null;

        if ("Manager".equals(type)) {
            Manager manager = new Manager(name, surname);
            manager.setLicenseNumber(specificInfo);
            staff = manager;
        } else if ("HouseKeeper".equals(type)) {
            HouseKeeper housekeeper = new HouseKeeper(name, surname);
            housekeeper.setYearsOfExperience(Integer.parseInt(specificInfo));
            staff = housekeeper;
        }

        if (staff != null) {
            staff.setStaffID(staffID);
            staff.setDateOfBirth(dateOfBirth);
            staffList.add(staff);
        }
    }
}
```

```
            System.out.println("Loaded " + staffList.size() + " staff members from " + STAFF_FILE);

        } catch (FileNotFoundException e) {
            System.out.println("Staff file not found. Starting with empty staff list.");
        } catch (IOException e) {
            System.err.println("Error loading staff data: " + e.getMessage());
        }

        return staffList;
    }

    // Save guests to text file
    public static void saveGuestsToTextFile(List<Guest> guestList) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(GUEST_FILE))) {
            writer.println("# Hotel Guest Data");
            writer.println("# Format: Name|Surname|RoomNumber|NightsStayed");

            for (Guest guest : guestList) {
                writer.printf("%s|%s|%d|%d%n",
                    guest.getName(),
                    guest.getSurname(),
                    guest.getRoomNumber(),
                    guest.getNightsStayed());
            }
```

```
        System.out.println("Guest data saved to " + GUEST_FILE);

    } catch (IOException e) {
        System.err.println("Error saving guest data: " + e.getMessage());
    }
}

// Load guests from text file
public static List<Guest> loadGuestsFromTextFile() {
    List<Guest> guestList = new ArrayList<>();

    try (BufferedReader reader = new BufferedReader(new FileReader(GUEST_FILE))) {
        String line;

        while ((line = reader.readLine()) != null) {
            if (line.startsWith("#") || line.trim().isEmpty()) {
                continue;
            }

            String[] parts = line.split("\\|");
            if (parts.length >= 4) {
                String name = parts[0];
                String surname = parts[1];
                int roomNumber = Integer.parseInt(parts[2]);
                int nightsStayed = Integer.parseInt(parts[3]);
```

```
            Guest guest = new Guest(name, surname);
            guest.setRoomNumber(roomNumber);
            guest.setNightsStayed(nightsStayed);
            guestList.add(guest);
          }
        }

        System.out.println("Loaded " + guestList.size() + " guests from " + GUEST_FILE);

      } catch (FileNotFoundException e) {
        System.out.println("Guest file not found. Starting with empty guest list.");
      } catch (IOException e) {
        System.err.println("Error loading guest data: " + e.getMessage());
      }

      return guestList;
    }

    // Binary serialization methods
    public static void saveAllDataToBinary(List<HotelStaff> staffList, List<Guest> guestList,
List<Room> roomList) {
      try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(BINARY_FILE))) {
        HotelData data = new HotelData(staffList, guestList, roomList);
        oos.writeObject(data);
        System.out.println("All data saved to binary file: " + BINARY_FILE);
```

```
    } catch (IOException e) {
        System.err.println("Error saving binary data: " + e.getMessage());
    }
  }

  public static HotelData loadAllDataFromBinary() {
    try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(BINARY_FILE))) {
        HotelData data = (HotelData) ois.readObject();
        System.out.println("All data loaded from binary file: " + BINARY_FILE);
        return data;

    } catch (FileNotFoundException e) {
        System.out.println("Binary file not found. Starting with empty data.");
        return new HotelData();
    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Error loading binary data: " + e.getMessage());
        return new HotelData();
    }
  }

  // Data container class for serialization
  public static class HotelData implements Serializable {
    private static final long serialVersionUID = 1L;

    private List<HotelStaff> staffList;
```

```
    private List<Guest> guestList;
    private List<Room> roomList;

    public HotelData() {
      this.staffList = new ArrayList<>();
      this.guestList = new ArrayList<>();
      this.roomList = new ArrayList<>();
    }

    public HotelData(List<HotelStaff> staffList, List<Guest> guestList, List<Room>
roomList) {
      this.staffList = new ArrayList<>(staffList);
      this.guestList = new ArrayList<>(guestList);
      this.roomList = new ArrayList<>(roomList);
    }

    // Getters
    public List<HotelStaff> getStaffList() { return staffList; }
    public List<Guest> getGuestList() { return guestList; }
    public List<Room> getRoomList() { return roomList; }
  }
}
```

Step 2: Make Room Classes Serializable

Modify all Room classes to implement Serializable:

```
// Add to Room.java
public abstract class Room implements Serializable {
   private static final long serialVersionUID = 1L;
   // ... rest of the code remains the same
}

// Add to StandardRoom.java
public class StandardRoom extends Room {
   private static final long serialVersionUID = 1L;
   // ... rest of the code remains the same
}

// Add to DeluxeRoom.java
public class DeluxeRoom extends Room {
   private static final long serialVersionUID = 1L;
   // ... rest of the code remains the same
}

// Add to Suite.java
public class Suite extends Room {
   private static final long serialVersionUID = 1L;
   // ... rest of the code remains the same
}
```

Step 3: Add File Operations to WestminsterHotelManager

Add these methods to WestminsterHotelManager class:

```
public void saveDataToFiles() {
    HotelDataManager.saveStaffToTextFile(hotelStaffList);
    HotelDataManager.saveGuestsToTextFile(hotelGuestList);
    HotelDataManager.saveAllDataToBinary(hotelStaffList, hotelGuestList, hotelRoomList);
    System.out.println("All data saved successfully!");
}

public void loadDataFromFiles() {
    // Load from text files
    List<HotelStaff> loadedStaff = HotelDataManager.loadStaffFromTextFile();
    List<Guest> loadedGuests = HotelDataManager.loadGuestsFromTextFile();

    if (!loadedStaff.isEmpty()) {
        hotelStaffList.clear();
        hotelStaffList.addAll(loadedStaff);
    }

    if (!loadedGuests.isEmpty()) {
        hotelGuestList.clear();
        hotelGuestList.addAll(loadedGuests);
    }

    System.out.println("Data loaded successfully!");
```

```
}

public void loadDataFromBinary() {
   HotelDataManager.HotelData data = HotelDataManager.loadAllDataFromBinary();

   if (!data.getStaffList().isEmpty()) {
      hotelStaffList.clear();
      hotelStaffList.addAll(data.getStaffList());
   }

   if (!data.getGuestList().isEmpty()) {
      hotelGuestList.clear();
      hotelGuestList.addAll(data.getGuestList());
   }

   if (!data.getRoomList().isEmpty()) {
      hotelRoomList.clear();
      hotelRoomList.addAll(data.getRoomList());
   }

   System.out.println("Binary data loaded successfully!");
```

QUESTION 3: Implement Exception Handling with Custom Exceptions

Task Description:

Create custom exception classes and implement comprehensive exception handling throughout the hotel management system.

Implementation:

Step 1: Create Base Exception Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/HotelManagementException.java`

```
package com.mycompany.hotelmanagementsystem;


public class HotelManagementException extends Exception {
    public HotelManagementException(String message) {
        super(message);
    }

    public HotelManagementException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Step 2: Create Specific Exception Classes

File Location:
`src/main/java/com/mycompany/hotelmanagementsystem/StaffLimitExceededException.java`

```
package com.mycompany.hotelmanagementsystem;

public class StaffLimitExceededException extends HotelManagementException {
   private final int currentCount;
   private final int maxLimit;

   public StaffLimitExceededException(int currentCount, int maxLimit) {
      super(String.format("Staff limit exceeded. Current: %d, Maximum: %d", currentCount, maxLimit));
      this.currentCount = currentCount;
      this.maxLimit = maxLimit;
   }

   public int getCurrentCount() { return currentCount; }
   public int getMaxLimit() { return maxLimit; }
}
```

File Location:
`src/main/java/com/mycompany/hotelmanagementsystem/InvalidRoomException.java`

```
package com.mycompany.hotelmanagementsystem;
```

```
public class InvalidRoomException extends HotelManagementException {
    private final int roomNumber;

    public InvalidRoomException(int roomNumber, String message) {
        super("Room " + roomNumber + ": " + message);
        this.roomNumber = roomNumber;
    }

    public int getRoomNumber() { return roomNumber; }
}
```

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/RoomNotAvailableException.java`

```
package com.mycompany.hotelmanagementsystem;

public class RoomNotAvailableException extends InvalidRoomException {
    public RoomNotAvailableException(int roomNumber) {
        super(roomNumber, "Room is not available for booking");
    }
}
```

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/InvalidStaffDataException.java`

```
package com.mycompany.hotelmanagementsystem;

public class InvalidStaffDataException extends HotelManagementException {
   private final String fieldName;
   private final String invalidValue;

   public InvalidStaffDataException(String fieldName, String invalidValue) {
      super(String.format("Invalid %s: '%s'", fieldName, invalidValue));
      this.fieldName = fieldName;
      this.invalidValue = invalidValue;
   }

   public String getFieldName() { return fieldName; }
   public String getInvalidValue() { return invalidValue; }
}
```

Step 3: Add Exception Handling to WestminsterHotelManager**

Modify the addHotelStaff method:

```
@Override
public void addHotelStaff() {
   Scanner s = new Scanner(System.in);

   try {
      if (hotelStaffList.size() >= staffLimit) {
         throw new StaffLimitExceededException(hotelStaffList.size(), staffLimit);
```

```
}

System.out.println("Press 1 if you want to add a Manager");
System.out.println("Press 2 if you want to add a Housekeeper");

int choiceStaff = getValidatedIntInput(s, "staff type choice", 1, 2);
s.nextLine(); // Consume newline

String name = getValidatedStringInput(s, "first name");
String surname = getValidatedStringInput(s, "last name");
String staffID = getValidatedStaffID(s);
LocalDate date = getValidatedDate(s);

HotelStaff staff = null;

switch (choiceStaff) {
  case 1:
    String licenseNum = getValidatedStringInput(s, "license number");
    Manager manager = new Manager(name, surname);
    manager.setLicenseNumber(licenseNum);
    manager.setDateOfBirth(date);
    manager.setStaffID(staffID);
    staff = manager;
    break;
```

```
case 2:
            int yearsOfExperience = getValidatedIntInput(s, "years of experience", 0, 50);
            HouseKeeper housekeeper = new HouseKeeper(name, surname);
            housekeeper.setYearsOfExperience(yearsOfExperience);
            housekeeper.setDateOfBirth(date);
            housekeeper.setStaffID(staffID);
            staff = housekeeper;
            break;
        }

        if (staff != null) {
          hotelStaffList.add(staff);
          System.out.println("Staff member added successfully!");
        }

    } catch (StaffLimitExceededException e) {
        System.err.println("Error: " + e.getMessage());
        System.out.println("Cannot add more staff members.");

    } catch (InvalidStaffDataException e) {
        System.err.println("Invalid input: " + e.getMessage());
        System.out.println("Please try again with valid data.");

    } catch (Exception e) {
        System.err.println("Unexpected error occurred: " + e.getMessage());
        e.printStackTrace();
```

```
    }
}

// Helper method for validated integer input
private int getValidatedIntInput(Scanner scanner, String fieldName, int min, int max)
        throws InvalidStaffDataException {
    System.out.println("Enter " + fieldName + " (" + min + "-" + max + "):");

    try {
        int value = scanner.nextInt();
        if (value < min || value > max) {
            throw new InvalidStaffDataException(fieldName,
                String.valueOf(value) + " (must be between " + min + " and " + max + ")");
        }
        return value;
    } catch (java.util.InputMismatchException e) {
        scanner.nextLine(); // Clear invalid input
        throw new InvalidStaffDataException(fieldName, "non-numeric input");
    }
}

// Helper method for validated string input
private String getValidatedStringInput(Scanner scanner, String fieldName)
        throws InvalidStaffDataException {
    System.out.println("Enter " + fieldName + ":");
    String input = scanner.nextLine().trim();
```

```
    if (input.isEmpty()) {
        throw new InvalidStaffDataException(fieldName, "empty string");
    }

    if (!input.matches("^[a-zA-Z\\s'-]+$")) {
        throw new InvalidStaffDataException(fieldName, input + " (contains invalid characters)");
    }

    if (input.length() > 50) {
        throw new InvalidStaffDataException(fieldName, input + " (too long, max 50 characters)");
    }

    return input;
}

// Helper method for validated staff ID
private String getValidatedStaffID(Scanner scanner) throws InvalidStaffDataException {
    System.out.println("Enter staff ID (format: S followed by 3-9 digits):");
    String staffID = scanner.nextLine().trim().toUpperCase();

    if (!staffID.matches("^S\\d{3,9}$")) {
        throw new InvalidStaffDataException("staff ID",
            staffID + " (must be S followed by 3-9 digits)");
    }
```

```
    // Check for duplicate staff ID
    for (HotelStaff existingStaff : hotelStaffList) {
        if (staffID.equals(existingStaff.getStaffID())) {
            throw new InvalidStaffDataException("staff ID", staffID + " (already exists)");
        }
    }

    return staffID;
}

// Helper method for validated date
private LocalDate getValidatedDate(Scanner scanner) throws InvalidStaffDataException {
    System.out.println("Enter date of birth (dd/MM/yyyy format only!):");
    String dateStr = scanner.nextLine().trim();

    try {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        LocalDate date = LocalDate.parse(dateStr, formatter);

        // Validate reasonable age range
        LocalDate now = LocalDate.now();
        int age = Period.between(date, now).getYears();

        if (age < 16) {
            throw new InvalidStaffDataException("date of birth",
```

```
            dateStr + " (employee must be at least 16 years old)");
        }

        if (age > 80) {
            throw new InvalidStaffDataException("date of birth",
                dateStr + " (employee cannot be more than 80 years old)");
        }

        return date;

    } catch (DateTimeParseException e) {
        throw new InvalidStaffDataException("date of birth",
            dateStr + " (invalid format, use dd/MM/yyyy)");
    }
}
```

QUESTION 4: Implement a Search and Filter System

Task Description:

Add functionality to search and filter staff, guests, and rooms based on various criteria using Java 8 Streams and Lambda expressions.

Implementation:

Step 1: Create HotelSearchManager Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/HotelSearchManager.java`

```
package com.mycompany.hotelmanagementsystem;

import java.time.LocalDate;
import java.time.Period;
import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class HotelSearchManager {

  // Staff search methods
  public static List<HotelStaff> searchStaffByName(List<HotelStaff> staffList, String name) {
    return staffList.stream()
      .filter(staff -> staff.getName().toLowerCase().contains(name.toLowerCase()) ||
              staff.getSurname().toLowerCase().contains(name.toLowerCase()))
      .collect(Collectors.toList());
  }

  public static List<HotelStaff> searchStaffById(List<HotelStaff> staffList, String staffId) {
    return staffList.stream()
      .filter(staff -> staff.getStaffID().equalsIgnoreCase(staffId))
      .collect(Collectors.toList());
  }
```

```
  public static List<HotelStaff> filterStaffByType(List<HotelStaff> staffList, Class<? extends HotelStaff> staffType) {
      return staffList.stream()
          .filter(staffType::isInstance)
          .collect(Collectors.toList());
  }

  public static List<HotelStaff> filterStaffByAge(List<HotelStaff> staffList, int minAge, int maxAge) {
      LocalDate now = LocalDate.now();
      return staffList.stream()
          .filter(staff -> {
              int age = Period.between(staff.getDateOfBirth(), now).getYears();
              return age >= minAge && age <= maxAge;
          })
          .collect(Collectors.toList());
  }

  public static List<HouseKeeper> filterHousekeepersByExperience(List<HotelStaff> staffList, int minExperience) {
      return staffList.stream()
          .filter(staff -> staff instanceof HouseKeeper)
          .map(staff -> (HouseKeeper) staff)
          .filter(housekeeper -> housekeeper.getYearsOfExperience() >= minExperience)
          .collect(Collectors.toList());
  }
```

```
    // Guest search methods
    public static List<Guest> searchGuestsByName(List<Guest> guestList, String name) {
        return guestList.stream()
            .filter(guest -> guest.getName().toLowerCase().contains(name.toLowerCase()) ||
                             guest.getSurname().toLowerCase().contains(name.toLowerCase()))
            .collect(Collectors.toList());
    }

    public static List<Guest> searchGuestsByRoom(List<Guest> guestList, int roomNumber)
{
        return guestList.stream()
            .filter(guest -> guest.getRoomNumber() == roomNumber)
            .collect(Collectors.toList());
    }

    public static List<Guest> filterGuestsByStayDuration(List<Guest> guestList, int
minNights, int maxNights) {
        return guestList.stream()
            .filter(guest -> guest.getNightsStayed() >= minNights && guest.getNightsStayed() <=
maxNights)
            .collect(Collectors.toList());
    }

    // Room search methods
    public static List<Room> searchRoomsByNumber(List<Room> roomList, int
roomNumber) {
```

```
    return roomList.stream()
        .filter(room -> room.getRoomNumber() == roomNumber)
        .collect(Collectors.toList());
  }

  public static List<Room> filterRoomsByType(List<Room> roomList, Class<? extends Room> roomType) {
    return roomList.stream()
        .filter(roomType::isInstance)
        .collect(Collectors.toList());
  }

  public static List<Room> filterRoomsByAvailability(List<Room> roomList, boolean available) {
    return roomList.stream()
        .filter(room -> room.isAvailable() == available)
        .collect(Collectors.toList());
  }

  public static List<Room> filterRoomsByPriceRange(List<Room> roomList, double minPrice, double maxPrice) {
    return roomList.stream()
        .filter(room -> room.getBasePrice() >= minPrice && room.getBasePrice() <= maxPrice)
        .collect(Collectors.toList());
  }
```

```
  public static List<Room> filterRoomsByCapacity(List<Room> roomList, int minCapacity)
{
    return roomList.stream()
      .filter(room -> room.getCapacity() >= minCapacity)
      .collect(Collectors.toList());
  }

  // Advanced search with multiple criteria
  public static List<HotelStaff> advancedStaffSearch(List<HotelStaff> staffList,
StaffSearchCriteria criteria) {
    return staffList.stream()
      .filter(staff -> {
        boolean matches = true;

        if (criteria.getName() != null && !criteria.getName().isEmpty()) {
          matches &=
staff.getName().toLowerCase().contains(criteria.getName().toLowerCase()) ||

staff.getSurname().toLowerCase().contains(criteria.getName().toLowerCase());
        }

        if (criteria.getStaffType() != null) {
          matches &= criteria.getStaffType().isInstance(staff);
        }

        if (criteria.getMinAge() > 0 || criteria.getMaxAge() > 0) {
          int age = Period.between(staff.getDateOfBirth(), LocalDate.now()).getYears();
```

```
            if (criteria.getMinAge() > 0) matches &= age >= criteria.getMinAge();
            if (criteria.getMaxAge() > 0) matches &= age <= criteria.getMaxAge();
        }

        if (criteria.getMinExperience() > 0 && staff instanceof HouseKeeper) {
            matches &= ((HouseKeeper) staff).getYearsOfExperience() >=
criteria.getMinExperience();
        }

        return matches;
      })
      .collect(Collectors.toList());
  }

  // Search criteria class
  public static class StaffSearchCriteria {
    private String name;
    private Class<? extends HotelStaff> staffType;
    private int minAge;
    private int maxAge;
    private int minExperience;

    public StaffSearchCriteria() {}

    // Getters and setters
    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }

    public Class<? extends HotelStaff> getStaffType() { return staffType; }
    public void setStaffType(Class<? extends HotelStaff> staffType) { this.staffType = staffType; }

    public int getMinAge() { return minAge; }
    public void setMinAge(int minAge) { this.minAge = minAge; }

    public int getMaxAge() { return maxAge; }
    public void setMaxAge(int maxAge) { this.maxAge = maxAge; }

    public int getMinExperience() { return minExperience; }
    public void setMinExperience(int minExperience) { this.minExperience = minExperience; }
  }
}
```

Step 2: Add Search Methods to WestminsterHotelManager

Add these methods to WestminsterHotelManager class:

```
public void searchStaff() {
  Scanner s = new Scanner(System.in);

  System.out.println("\n=== STAFF SEARCH ===");
  System.out.println("1. Search by Name");
```

```
System.out.println("2. Search by Staff ID");
System.out.println("3. Filter by Staff Type");
System.out.println("4. Filter by Age Range");
System.out.println("5. Filter Housekeepers by Experience");
System.out.println("6. Advanced Search");

System.out.print("Choose search option: ");
int choice = s.nextInt();
s.nextLine(); // Consume newline

List<HotelStaff> results = new ArrayList<>();

try {
   switch (choice) {
      case 1:
         System.out.print("Enter name to search: ");
         String name = s.nextLine();
         results = HotelSearchManager.searchStaffByName(hotelStaffList, name);
         break;

      case 2:
         System.out.print("Enter Staff ID: ");
         String staffId = s.nextLine();
         results = HotelSearchManager.searchStaffById(hotelStaffList, staffId);
         break;
```

```
        case 3:
            System.out.println("1. Managers only");
            System.out.println("2. Housekeepers only");
            System.out.print("Choose type: ");
            int typeChoice = s.nextInt();

            if (typeChoice == 1) {
                results = HotelSearchManager.filterStaffByType(hotelStaffList, Manager.class);
            } else if (typeChoice == 2) {
                results = HotelSearchManager.filterStaffByType(hotelStaffList,
HouseKeeper.class);
            }
            break;

        case 4:
            System.out.print("Enter minimum age: ");
            int minAge = s.nextInt();
            System.out.print("Enter maximum age: ");
            int maxAge = s.nextInt();
            results = HotelSearchManager.filterStaffByAge(hotelStaffList, minAge, maxAge);
            break;

        case 5:
            System.out.print("Enter minimum years of experience: ");
            int minExp = s.nextInt();
            List<HouseKeeper> housekeepers =
HotelSearchManager.filterHousekeepersByExperience(hotelStaffList, minExp);
```

```
            results = new ArrayList<>(housekeepers);
            break;

        case 6:
            results = performAdvancedStaffSearch(s);
            break;

        default:
            System.out.println("Invalid choice");
            return;
      }

      displaySearchResults("Staff", results);

    } catch (Exception e) {
      System.err.println("Error during search: " + e.getMessage());
    }
}

private List<HotelStaff> performAdvancedStaffSearch(Scanner s) {
    HotelSearchManager.StaffSearchCriteria criteria = new
HotelSearchManager.StaffSearchCriteria();

    System.out.print("Enter name (or press Enter to skip): ");
    String name = s.nextLine().trim();
    if (!name.isEmpty()) {
```

```
    criteria.setName(name);
}

System.out.print("Filter by staff type? (1=Manager, 2=Housekeeper, 0=No filter): ");
int typeChoice = s.nextInt();
if (typeChoice == 1) {
    criteria.setStaffType(Manager.class);
} else if (typeChoice == 2) {
    criteria.setStaffType(HouseKeeper.class);
}

System.out.print("Enter minimum age (0 to skip): ");
int minAge = s.nextInt();
if (minAge > 0) {
    criteria.setMinAge(minAge);
}

System.out.print("Enter maximum age (0 to skip): ");
int maxAge = s.nextInt();
if (maxAge > 0) {
    criteria.setMaxAge(maxAge);
}

System.out.print("Enter minimum experience for housekeepers (0 to skip): ");
int minExp = s.nextInt();
if (minExp > 0) {
```

```
        criteria.setMinExperience(minExp);
    }

    return HotelSearchManager.advancedStaffSearch(hotelStaffList, criteria);
}

private <T> void displaySearchResults(String type, List<T> results) {
    System.out.println("\n=== SEARCH RESULTS ===");

    if (results.isEmpty()) {
        System.out.println("No " + type.toLowerCase() + "s found matching the criteria.");
    } else {
        System.out.println("Found " + results.size() + " " + type.toLowerCase() + "(s):");
        System.out.println("-".repeat(50));

        for (T item : results) {
            System.out.println(item.toString());
        }
    }

    System.out.println("=".repeat(50));
}
```

QUESTION 5: Implement a Booking Management System with Date Handling

Task Description:

Create a comprehensive booking system that handles check-in/check-out dates, calculates stay duration, and manages room availability with proper date validation.

Implementation:

Step 1: Create Booking Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/Booking.java`

```
package com.mycompany.hotelmanagementsystem;

import java.io.Serializable;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Booking implements Serializable {
    private static final long serialVersionUID = 1L;
    private static int nextBookingId = 1000;

    private int bookingId;
    private Guest guest;
    private Room room;
    private LocalDate checkInDate;
    private LocalDate checkOutDate;
    private int numberOfNights;
```

```
    private double totalAmount;
    private BookingStatus status;
    private LocalDate bookingDate;
    private String specialRequests;

    public Booking(Guest guest, Room room, LocalDate checkInDate, LocalDate
checkOutDate) {
        this.bookingId = nextBookingId++;
        this.guest = guest;
        this.room = room;
        this.checkInDate = checkInDate;
        this.checkOutDate = checkOutDate;
        this.numberOfNights = (int) ChronoUnit.DAYS.between(checkInDate, checkOutDate);
        this.totalAmount = room.calculateTotalPrice(numberOfNights);
        this.status = BookingStatus.CONFIRMED;
        this.bookingDate = LocalDate.now();
        this.specialRequests = "";
    }

    // Getters and Setters
    public int getBookingId() { return bookingId; }

    public Guest getGuest() { return guest; }
    public void setGuest(Guest guest) { this.guest = guest; }

    public Room getRoom() { return room; }
```

```
    public void setRoom(Room room) { this.room = room; }

    public LocalDate getCheckInDate() { return checkInDate; }
    public void setCheckInDate(LocalDate checkInDate) {
        this.checkInDate = checkInDate;
        updateNightsAndAmount();
    }

    public LocalDate getCheckOutDate() { return checkOutDate; }
    public void setCheckOutDate(LocalDate checkOutDate) {
        this.checkOutDate = checkOutDate;
        updateNightsAndAmount();
    }

    public int getNumberOfNights() { return numberOfNights; }

    public double getTotalAmount() { return totalAmount; }

    public BookingStatus getStatus() { return status; }
    public void setStatus(BookingStatus status) { this.status = status; }

    public LocalDate getBookingDate() { return bookingDate; }

    public String getSpecialRequests() { return specialRequests; }
    public void setSpecialRequests(String specialRequests) { this.specialRequests =
specialRequests; }
```

```
  private void updateNightsAndAmount() {
    if (checkInDate != null && checkOutDate != null) {
      this.numberOfNights = (int) ChronoUnit.DAYS.between(checkInDate,
checkOutDate);
      this.totalAmount = room.calculateTotalPrice(numberOfNights);
    }
  }

  public boolean isActive() {
    return status == BookingStatus.CONFIRMED || status == BookingStatus.CHECKED_IN;
  }

  public boolean isCurrentlyOccupied() {
    LocalDate today = LocalDate.now();
    return status == BookingStatus.CHECKED_IN &&
        !today.isBefore(checkInDate) &&
        today.isBefore(checkOutDate);
  }

  public String getFormattedCheckInDate() {
    return checkInDate.format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));
  }

  public String getFormattedCheckOutDate() {
    return checkOutDate.format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));
```

```
    }

    @Override
    public String toString() {
        return String.format("Booking ID: %d | Guest: %s %s | Room: %d (%s) | %s to %s (%d nights) | $%.2f | Status: %s",
                bookingId, guest.getName(), guest.getSurname(), room.getRoomNumber(), room.getRoomType(),
                getFormattedCheckInDate(), getFormattedCheckOutDate(), numberOfNights, totalAmount, status);
    }
}
```

Step 2: Create BookingStatus Enum

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/BookingStatus.java`

```
package com.mycompany.hotelmanagementsystem;

public enum BookingStatus {
    CONFIRMED("Confirmed"),
    CHECKED_IN("Checked In"),
    CHECKED_OUT("Checked Out"),
    CANCELLED("Cancelled"),
    NO_SHOW("No Show");

    private final String displayName;
```

```
    BookingStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

Step 3: Create BookingManager Class

File Location: `src/main/java/com/mycompany/hotelmanagementsystem/BookingManager.java`

```
package com.mycompany.hotelmanagementsystem;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
```

```
public class BookingManager {
   private List<Booking> bookingList;
   private DateTimeFormatter dateFormatter;

   public BookingManager() {
      this.bookingList = new ArrayList<>();
      this.dateFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
   }

   public Booking createBooking(Guest guest, Room room, LocalDate checkInDate,
LocalDate checkOutDate)
         throws InvalidRoomException {

      // Validate dates
      validateBookingDates(checkInDate, checkOutDate);

      // Check room availability
      if (!isRoomAvailable(room, checkInDate, checkOutDate)) {
         throw new RoomNotAvailableException(room.getRoomNumber());
      }

      // Create booking
      Booking booking = new Booking(guest, room, checkInDate, checkOutDate);
      bookingList.add(booking);
```

```
    // Update room status
    room.setAvailable(false);

    return booking;
}

public void checkInGuest(int bookingId) throws InvalidRoomException {
    Booking booking = findBookingById(bookingId);

    if (booking == null) {
        throw new InvalidRoomException(0, "Booking not found with ID: " + bookingId);
    }

    if (booking.getStatus() != BookingStatus.CONFIRMED) {
        throw new InvalidRoomException(booking.getRoom().getRoomNumber(),
            "Cannot check in - booking status is " + booking.getStatus());
    }

    LocalDate today = LocalDate.now();
    if (today.isBefore(booking.getCheckInDate())) {
        throw new InvalidRoomException(booking.getRoom().getRoomNumber(),
            "Cannot check in before check-in date: " + booking.getFormattedCheckInDate());
    }

    booking.setStatus(BookingStatus.CHECKED_IN);
    System.out.println("Guest checked in successfully!");
```

```
    System.out.println("Booking: " + booking);
}

public void checkOutGuest(int bookingId) throws InvalidRoomException {
    Booking booking = findBookingById(bookingId);

    if (booking == null) {
        throw new InvalidRoomException(0, "Booking not found with ID: " + bookingId);
    }

    if (booking.getStatus() != BookingStatus.CHECKED_IN) {
        throw new InvalidRoomException(booking.getRoom().getRoomNumber(),
            "Cannot check out - guest is not checked in");
    }

    booking.setStatus(BookingStatus.CHECKED_OUT);
    booking.getRoom().setAvailable(true);

    System.out.println("Guest checked out successfully!");
    System.out.printf("Final bill: $%.2f%n", booking.getTotalAmount());
}

public void cancelBooking(int bookingId) throws InvalidRoomException {
    Booking booking = findBookingById(bookingId);

    if (booking == null) {
```

```
            throw new InvalidRoomException(0, "Booking not found with ID: " + bookingId);
        }

        if (booking.getStatus() == BookingStatus.CHECKED_IN) {
            throw new InvalidRoomException(booking.getRoom().getRoomNumber(),
                "Cannot cancel - guest is currently checked in");
        }

        booking.setStatus(BookingStatus.CANCELLED);
        booking.getRoom().setAvailable(true);

        System.out.println("Booking cancelled successfully!");
    }

    public boolean isRoomAvailable(Room room, LocalDate checkInDate, LocalDate
checkOutDate) {
        return bookingList.stream()
            .filter(booking -> booking.getRoom().getRoomNumber() == room.getRoomNumber())
            .filter(booking -> booking.isActive())
            .noneMatch(booking -> datesOverlap(
                booking.getCheckInDate(), booking.getCheckOutDate(),
                checkInDate, checkOutDate));
    }

    private boolean datesOverlap(LocalDate start1, LocalDate end1, LocalDate start2,
LocalDate end2) {
        return start1.isBefore(end2) && start2.isBefore(end1);
```

```
  }

  private void validateBookingDates(LocalDate checkInDate, LocalDate checkOutDate)
      throws InvalidRoomException {

    LocalDate today = LocalDate.now();

    if (checkInDate.isBefore(today)) {
      throw new InvalidRoomException(0, "Check-in date cannot be in the past");
    }

    if (checkOutDate.isBefore(checkInDate) || checkOutDate.isEqual(checkInDate)) {
      throw new InvalidRoomException(0, "Check-out date must be after check-in date");
    }

    if (checkInDate.isAfter(today.plusYears(1))) {
      throw new InvalidRoomException(0, "Check-in date cannot be more than 1 year in
advance");
    }
  }

  public Booking findBookingById(int bookingId) {
    return bookingList.stream()
      .filter(booking -> booking.getBookingId() == bookingId)
      .findFirst()
      .orElse(null);
```

```
    }

    public List<Booking> getBookingsByGuest(String guestName) {
        return bookingList.stream()
                .filter(booking ->
                        booking.getGuest().getName().toLowerCase().contains(guestName.toLowerCase())
||

booking.getGuest().getSurname().toLowerCase().contains(guestName.toLowerCase()))
                .collect(Collectors.toList());
    }

    public List<Booking> getBookingsByRoom(int roomNumber) {
        return bookingList.stream()
                .filter(booking -> booking.getRoom().getRoomNumber() == roomNumber)
                .collect(Collectors.toList());
    }

    public List<Booking> getBookingsByStatus(BookingStatus status) {
        return bookingList.stream()
                .filter(booking -> booking.getStatus() == status)
                .collect(Collectors.toList());
    }

    public List<Booking> getCurrentBookings() {
        return bookingList.stream()
                .filter(booking -> booking.isCurrentlyOccupied())
```

```
        .collect(Collectors.toList());
    }

    public List<Booking> getUpcomingBookings() {
        LocalDate today = LocalDate.now();
        return bookingList.stream()
            .filter(booking -> booking.getStatus() == BookingStatus.CONFIRMED)
            .filter(booking -> booking.getCheckInDate().isAfter(today))
            .collect(Collectors.toList());
    }

    public double getTotalRevenue() {
        return bookingList.stream()
            .filter(booking -> booking.getStatus() == BookingStatus.CHECKED_OUT)
            .mapToDouble(Booking::getTotalAmount)
            .sum();
    }

    public double getPendingRevenue() {
        return bookingList.stream()
            .filter(booking -> booking.getStatus() == BookingStatus.CONFIRMED ||
                               booking.getStatus() == BookingStatus.CHECKED_IN)
            .mapToDouble(Booking::getTotalAmount)
            .sum();
    }
```

```
public void printBookingReport() {
    System.out.println("\n=== BOOKING REPORT ===");
    System.out.println("Total Bookings: " + bookingList.size());
    System.out.println("Current Bookings: " + getCurrentBookings().size());
    System.out.println("Upcoming Bookings: " + getUpcomingBookings().size());
    System.out.printf("Total Revenue: $%.2f%n", getTotalRevenue());
    System.out.printf("Pending Revenue: $%.2f%n", getPendingRevenue());

    System.out.println("\nBookings by Status:");
    for (BookingStatus status : BookingStatus.values()) {
        long count = bookingList.stream()
            .filter(booking -> booking.getStatus() == status)
            .count();
        System.out.println("  " + status + ": " + count);
    }
    System.out.println("=".repeat(50));
}

public List<Booking> getAllBookings() {
    return new ArrayList<>(bookingList);
}

public void setBookingList(List<Booking> bookingList) {
    this.bookingList = new ArrayList<>(bookingList);
}
```

```
    public LocalDate parseDate(String dateString) throws DateTimeParseException {
        return LocalDate.parse(dateString, dateFormatter);
    }

    public String formatDate(LocalDate date) {
        return date.format(dateFormatter);
    }
}
```

Step 4: Add Booking Methods to WestminsterHotelManager

Add these methods to WestminsterHotelManager class:

```
// Add this field to the class
private BookingManager bookingManager;

// Update constructor
public WestminsterHotelManager(int maxMembersNumber) {
    hotelStaffList = new ArrayList<>();
    hotelGuestList = new ArrayList<>();
    hotelRoomList = new ArrayList<>();
    bookingManager = new BookingManager(); // Add this line
    staffLimit = maxMembersNumber;
    initializeRooms();
}
```

```
public void createBooking() {
    Scanner s = new Scanner(System.in);

    try {
        // Get guest information
        System.out.println("\n=== CREATE NEW BOOKING ===");
        System.out.print("Enter guest first name: ");
        String firstName = s.nextLine().trim();
        System.out.print("Enter guest last name: ");
        String lastName = s.nextLine().trim();

        Guest guest = new Guest(firstName, lastName);

        // Show available rooms
        printAvailableRooms();

        System.out.print("Enter room number: ");
        int roomNumber = s.nextInt();
        s.nextLine(); // Consume newline

        Room selectedRoom = findRoomByNumber(roomNumber);
        if (selectedRoom == null) {
            System.out.println("Room not found!");
            return;
        }
```

```
// Get dates
        System.out.print("Enter check-in date (dd/MM/yyyy): ");
        String checkInStr = s.nextLine();
        LocalDate checkInDate = bookingManager.parseDate(checkInStr);

        System.out.print("Enter check-out date (dd/MM/yyyy): ");
        String checkOutStr = s.nextLine();
        LocalDate checkOutDate = bookingManager.parseDate(checkOutStr);

        // Get special requests
        System.out.print("Enter special requests (optional): ");
        String specialRequests = s.nextLine();

        // Create booking
        Booking booking = bookingManager.createBooking(guest, selectedRoom, checkInDate,
checkOutDate);
        booking.setSpecialRequests(specialRequests);

        // Add guest to guest list
        hotelGuestList.add(guest);

        System.out.println("\nBooking created successfully!");
        System.out.println(booking);
        System.out.printf("Total amount: $%.2f%n", booking.getTotalAmount());
```

```
    } catch (DateTimeParseException e) {
        System.err.println("Invalid date format. Please use dd/MM/yyyy");
    } catch (InvalidRoomException e) {
        System.err.println("Booking Error: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Error creating booking: " + e.getMessage());
    }
}

public void checkInGuest() {
    Scanner s = new Scanner(System.in);

    try {
        System.out.println("\n=== CHECK IN GUEST ===");
        System.out.print("Enter booking ID: ");
        int bookingId = s.nextInt();

        bookingManager.checkInGuest(bookingId);

    } catch (InvalidRoomException e) {
        System.err.println("Check-in Error: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Error during check-in: " + e.getMessage());
    }
}
```

```
public void checkOutGuest() {
    Scanner s = new Scanner(System.in);

    try {
        System.out.println("\n=== CHECK OUT GUEST ===");
        System.out.print("Enter booking ID: ");
        int bookingId = s.nextInt();

        bookingManager.checkOutGuest(bookingId);

    } catch (InvalidRoomException e) {
        System.err.println("Check-out Error: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Error during check-out: " + e.getMessage());
    }
}

public void viewBookings() {
    Scanner s = new Scanner(System.in);

    System.out.println("\n=== VIEW BOOKINGS ===");
    System.out.println("1. All Bookings");
    System.out.println("2. Current Bookings");
    System.out.println("3. Upcoming Bookings");
    System.out.println("4. Bookings by Guest");
    System.out.println("5. Bookings by Room");
```

```
System.out.println("6. Bookings by Status");

System.out.print("Choose option: ");
int choice = s.nextInt();
s.nextLine(); // Consume newline

List<Booking> bookings = new ArrayList<>();

try {
   switch (choice) {
      case 1:
         bookings = bookingManager.getAllBookings();
         break;
      case 2:
         bookings = bookingManager.getCurrentBookings();
         break;
      case 3:
         bookings = bookingManager.getUpcomingBookings();
         break;
      case 4:
         System.out.print("Enter guest name: ");
         String guestName = s.nextLine();
         bookings = bookingManager.getBookingsByGuest(guestName);
         break;
      case 5:
         System.out.print("Enter room number: ");
```

```
            int roomNumber = s.nextInt();
            bookings = bookingManager.getBookingsByRoom(roomNumber);
            break;
        case 6:
            System.out.println("Select status:");
            BookingStatus[] statuses = BookingStatus.values();
            for (int i = 0; i < statuses.length; i++) {
                System.out.println((i + 1) + ". " + statuses[i]);
            }
            System.out.print("Choose status: ");
            int statusChoice = s.nextInt();
            if (statusChoice >= 1 && statusChoice <= statuses.length) {
                bookings = bookingManager.getBookingsByStatus(statuses[statusChoice - 1]);
            }
            break;
        default:
            System.out.println("Invalid choice");
            return;
      }

      displayBookings(bookings);

    } catch (Exception e) {
      System.err.println("Error viewing bookings: " + e.getMessage());
    }
}
```

```
private void displayBookings(List<Booking> bookings) {
    System.out.println("\n=== BOOKING LIST ===");

    if (bookings.isEmpty()) {
        System.out.println("No bookings found.");
    } else {
        System.out.println("Found " + bookings.size() + " booking(s):");
        System.out.println("-".repeat(100));

        for (Booking booking : bookings) {
            System.out.println(booking);
            if (!booking.getSpecialRequests().isEmpty()) {
                System.out.println("  Special Requests: " + booking.getSpecialRequests());
            }
            System.out.println();
        }
    }

    System.out.println("=".repeat(100));
}

private Room findRoomByNumber(int roomNumber) {
    return hotelRoomList.stream()
        .filter(room -> room.getRoomNumber() == roomNumber)
        .findFirst()
```

```
        .orElse(null);
}

public void showBookingReport() {
    bookingManager.printBookingReport();
}
```

Object-Oriented Programming, Threading, and Logging Concepts

Q1: How would you add a new staff type "Receptionist" to the system?

To add a Receptionist class:

```
public class Receptionist extends HotelStaff {
private String[] languagesSpoken;
private boolean hasComputerSkills;
public Receptionist(String name, String surname) {
super(name, surname);
}
// Getters and setters
public void setLanguagesSpoken(String[] languages) {
this.languagesSpoken = languages;
}
public void setHasComputerSkills(boolean hasSkills) {
this.hasComputerSkills = hasSkills;
}
@Override
```

```
public String toString() {
return super.toString() + " Receptionist - Languages: " +
Arrays.toString(languagesSpoken) + ", Computer Skills: " + hasComputerSkills;
}
}
```

Then update the switch statement in addHotelStaff() to include case 3 for Receptionist.

Q2: How would you implement a search functionality to find staff by ID?

Implementation:

```
public HotelStaff findStaffByID(String staffID) {
for (HotelStaff staff : hotelStaffList) {
if (staff.getStaffID().equals(staffID)) {
return staff;
}
}
return null; // Not found
}
// Alternative using streams (Java 8+):
public Optional<HotelStaff> findStaffByIDStream(String staffID) {
return hotelStaffList.stream()
.filter(staff -> staff.getStaffID().equals(staffID))
.findFirst();
}
```

Q3: How would you sort the staff list by name?

Several approaches:

```
// Using Collections.sort with Comparator
Collections.sort(hotelStaffList, new Comparator<HotelStaff>() {
@Override
public int compare(HotelStaff s1, HotelStaff s2) {
return s1.getName().compareTo(s2.getName());
}
});
// Using lambda expression (Java 8+):
Collections.sort(hotelStaffList, (s1, s2) -> s1.getName().compareTo(s2.getName()));
// Using method reference:
hotelStaffList.sort(Comparator.comparing(HotelStaff::getName));
```

Q4: How would you add exception handling for duplicate staff IDs?

Implementation:

```
public class DuplicateStaffIDException extends Exception {
public DuplicateStaffIDException(String message) {
super(message);
}
}
public void addStaffToList(HotelStaff staff) throws DuplicateStaffIDException {
// Check for duplicate ID
for (HotelStaff existing : hotelStaffList) {
if (existing.getStaffID().equals(staff.getStaffID())) {
```

```
throw new DuplicateStaffIDException("Staff ID " + staff.getStaffID() + " already exists");
}
}
if (this.hotelStaffList.size() < staffLimit) {
hotelStaffList.add(staff);
} else {
throw new IllegalStateException("No more space in the list");
}
}
```

THREADING CONCEPTS

Q5: How would you make the hotel management system thread-safe?

Several approaches:

```
// 1. Synchronized methods
public synchronized void addStaffToList(HotelStaff staff) {
if (this.hotelStaffList.size() < staffLimit) {
hotelStaffList.add(staff);
}
}
// 2. Synchronized blocks
public void printHotelStaffList() {
synchronized(hotelStaffList) {
if (!hotelStaffList.isEmpty()) {
for (HotelStaff member : hotelStaffList) {
System.out.println(member.toString());
}
```

```
}
}
}
// 3. Use concurrent collections
private ConcurrentHashMap<String, HotelStaff> staffMap;
private CopyOnWriteArrayList<HotelStaff> hotelStaffList;
```

Q6: Implement a background thread to automatically save hotel data every 5 minutes.

Implementation:

```
public class AutoSaveThread extends Thread {
private WestminsterHotelManager hotelManager;
private volatile boolean running = true;
public AutoSaveThread(WestminsterHotelManager manager) {
this.hotelManager = manager;
setDaemon(true); // Dies when main program ends
}
@Override
public void run() {
while (running) {
try {
Thread.sleep(5 * 60 * 1000); // 5 minutes
hotelManager.saveToFile();
System.out.println("Auto-save completed at " + LocalDateTime.now());
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
```

```
}
}
}
public void stopAutoSave() {
running = false;
interrupt();
}
}
```

Q7: How would you implement a producer-consumer pattern for guest check-ins?

Implementation using BlockingQueue:

```
public class GuestCheckInSystem {
private BlockingQueue<Guest> checkInQueue = new LinkedBlockingQueue<>();
// Producer - adds guests to queue
class CheckInProducer implements Runnable {
@Override
public void run() {
while (true) {
try {
Guest guest = receiveGuestFromReception();
checkInQueue.put(guest);
System.out.println("Guest queued for check-in: " + guest.getName());
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
```

```
}
}
}
}
// Consumer - processes check-ins
class CheckInProcessor implements Runnable {
@Override
public void run() {
while (true) {
try {
Guest guest = checkInQueue.take();
processCheckIn(guest);
System.out.println("Processed check-in for: " + guest.getName());
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
}
}
```

Q8: How would you implement a thread pool for handling multiple guest requests?

Implementation:

```
public class HotelRequestHandler {
private ExecutorService threadPool;
```

```
public HotelRequestHandler(int poolSize) {
threadPool = Executors.newFixedThreadPool(poolSize);
}
public void handleGuestRequest(Guest guest, String requestType) {
threadPool.submit(new GuestRequestTask(guest, requestType));
}
private class GuestRequestTask implements Runnable {
private Guest guest;
private String requestType;
public GuestRequestTask(Guest guest, String requestType) {
this.guest = guest;
this.requestType = requestType;
}
@Override
public void run() {
System.out.println("Processing " + requestType + " for " + guest.getName());
// Process the request
processRequest(guest, requestType);
}
}
public void shutdown() {
threadPool.shutdown();
}
}
```

Q9: How would you implement logging in the hotel management system using java.util.logging?

Implementation:

```
import java.util.logging.*;
import java.io.IOException;
public class WestminsterHotelManager implements HotelManager {
private static final Logger logger =
Logger.getLogger(WestminsterHotelManager.class.getName());
static {
try {
// Configure file handler
FileHandler fileHandler = new FileHandler("hotel_management.log", true);
fileHandler.setFormatter(new SimpleFormatter());
logger.addHandler(fileHandler);
// Configure console handler
ConsoleHandler consoleHandler = new ConsoleHandler();
consoleHandler.setFormatter(new SimpleFormatter());
logger.addHandler(consoleHandler);
logger.setLevel(Level.ALL);
} catch (IOException e) {
logger.severe("Failed to setup logger: " + e.getMessage());
}
}
@Override
public void addHotelStaff() {
```

```
logger.info("Starting to add new hotel staff");
try {
// existing code
logger.info("Successfully added staff member: " + staff.getName());
} catch (Exception e) {
logger.severe("Error adding staff: " + e.getMessage());
}
}
}
```

Q10: How would you implement custom log formatting for the hotel system?

Custom formatter implementation:

```
public class HotelLogFormatter extends Formatter {
private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
@Override
public String format(LogRecord record) {
StringBuilder sb = new StringBuilder();
// Timestamp
sb.append("[").append(LocalDateTime.now().format(dateFormatter)).append("] ");
// Level
sb.append("[").append(record.getLevel()).append("] ");
// Class and method
sb.append("[").append(record.getSourceClassName()).append(".");
sb.append(record.getSourceMethodName()).append("] ");
// Message
```

```
sb.append(record.getMessage());
// Exception if present
if (record.getThrown() != null) {
sb.append("\nException: ").append(record.getThrown().toString());
}
sb.append("\n");
return sb.toString();
}
}
// Usage:
FileHandler fileHandler = new FileHandler("hotel.log", true);
fileHandler.setFormatter(new HotelLogFormatter());
logger.addHandler(fileHandler);
```

Q11: How would you implement different log files for different components?

Implementation:

```
public class HotelLoggingManager {
private static final Logger staffLogger = Logger.getLogger("hotel.staff");
private static final Logger guestLogger = Logger.getLogger("hotel.guest");
private static final Logger systemLogger = Logger.getLogger("hotel.system");
static {
try {
// Staff operations log
FileHandler staffHandler = new FileHandler("staff_operations.log", true);
staffHandler.setFormatter(new SimpleFormatter());
staffLogger.addHandler(staffHandler);
```

```
staffLogger.setUseParentHandlers(false);
// Guest operations log
FileHandler guestHandler = new FileHandler("guest_operations.log", true);
guestHandler.setFormatter(new SimpleFormatter());
guestLogger.addHandler(guestHandler);
guestLogger.setUseParentHandlers(false);
// System log
FileHandler systemHandler = new FileHandler("system.log", true);
systemHandler.setFormatter(new SimpleFormatter());
systemLogger.addHandler(systemHandler);
systemLogger.setUseParentHandlers(false);
} catch (IOException e) {
System.err.println("Failed to setup loggers: " + e.getMessage());
}
}
public static Logger getStaffLogger() { return staffLogger; }
public static Logger getGuestLogger() { return guestLogger; }
public static Logger getSystemLogger() { return systemLogger; }
}
```

Q12: How would you implement log rotation and archiving?

Implementation:

```
public class RotatingLogHandler extends Handler {
private String baseFileName;
private int maxFileSize;
```

```
private int maxFiles;
private FileHandler currentHandler;
private int currentFileSize;
public RotatingLogHandler(String baseFileName, int maxFileSize, int maxFiles) {
this.baseFileName = baseFileName;
this.maxFileSize = maxFileSize;
this.maxFiles = maxFiles;
createNewLogFile();
}
@Override
public void publish(LogRecord record) {
if (currentHandler != null) {
currentHandler.publish(record);
currentFileSize += getFormatter().format(record).length();
if (currentFileSize >= maxFileSize) {
rotateLogFiles();
}
}
}
private void rotateLogFiles() {
currentHandler.close();
// Move existing files
for (int i = maxFiles - 1; i > 0; i--) {
File oldFile = new File(baseFileName + "." + i);
File newFile = new File(baseFileName + "." + (i + 1));
if (oldFile.exists()) {
```

```
oldFile.renameTo(newFile);
}
}
// Move current log to .1
File currentFile = new File(baseFileName);
File firstBackup = new File(baseFileName + ".1");
currentFile.renameTo(firstBackup);
createNewLogFile();
}
private void createNewLogFile() {
try {
currentHandler = new FileHandler(baseFileName, true);
currentHandler.setFormatter(getFormatter());
currentFileSize = 0;
} catch (IOException e) {
System.err.println("Failed to create new log file: " + e.getMessage());
}
}
}
```

DESIGN PATTERNS

Q13: How would you implement the Singleton pattern for the hotel manager?

Implementation:

```
public class WestminsterHotelManager implements HotelManager {
private static volatile WestminsterHotelManager instance;
private static final Object lock = new Object();
private WestminsterHotelManager(int maxMembersNumber) {
hotelStaffList = new ArrayList<>();
hotelGuestList = new ArrayList<>();
staffLimit = maxMembersNumber;
}
public static WestminsterHotelManager getInstance(int maxMembersNumber) {
if (instance == null) {
synchronized (lock) {
if (instance == null) {
instance = new WestminsterHotelManager(maxMembersNumber);
}
}
}
return instance;
}
public static WestminsterHotelManager getInstance() {
return getInstance(100); // Default capacity
}
}
```

Q14: How would you implement the Observer pattern for staff changes?

Implementation:

```
interface StaffObserver {
void onStaffAdded(HotelStaff staff);
void onStaffRemoved(HotelStaff staff);
void onStaffUpdated(HotelStaff staff);
}
public class WestminsterHotelManager implements HotelManager {
private List<StaffObserver> observers = new ArrayList<>();
public void addObserver(StaffObserver observer) {
observers.add(observer);
}
public void removeObserver(StaffObserver observer) {
observers.remove(observer);
}
private void notifyStaffAdded(HotelStaff staff) {
for (StaffObserver observer : observers) {
observer.onStaffAdded(staff);
}
}
@Override
public void addStaffToList(HotelStaff staff) {
if (this.hotelStaffList.size() < staffLimit) {
hotelStaffList.add(staff);
notifyStaffAdded(staff);
}
```

```
}
}
// Usage:
public class AuditLogger implements StaffObserver {
@Override
public void onStaffAdded(HotelStaff staff) {
System.out.println("AUDIT: Staff added - " + staff.getName());
}
}
```

Q28: Implement the Command pattern for hotel operations.

A28: Implementation:

Java

```
interface Command {
void execute();
void undo();
}
class AddStaffCommand implements Command {
private WestminsterHotelManager manager;
private HotelStaff staff;
private boolean executed = false;
public AddStaffCommand(WestminsterHotelManager manager, HotelStaff staff) {
this.manager = manager;
this.staff = staff;
}
@Override
public void execute() {
```

```
manager.addStaffToList(staff);
executed = true;
}
@Override
public void undo() {
if (executed) {
manager.removeStaff(staff);
executed = false;
}
}
}
class CommandManager {
private Stack<Command> commandHistory = new Stack<>();
public void executeCommand(Command command) {
command.execute();
commandHistory.push(command);
}
public void undoLastCommand() {
if (!commandHistory.isEmpty()) {
Command lastCommand = commandHistory.pop();
lastCommand.undo();
}
}
}
```

FILE I/O AND SERIALIZATION

Q15: How would you implement data persistence using file I/O?

Implementation using serialization:

```
import java.io.*;
import java.util.ArrayList;
public class DataPersistence {
private static final String STAFF_FILE = "hotel_staff.dat";
private static final String GUEST_FILE = "hotel_guests.dat";
public void saveStaffData(ArrayList<HotelStaff> staffList) {
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(STAFF_FILE))) {
oos.writeObject(staffList);
System.out.println("Staff data saved successfully");
} catch (IOException e) {
System.err.println("Error saving staff data: " + e.getMessage());
}
}
@SuppressWarnings("unchecked")
public ArrayList<HotelStaff> loadStaffData() {
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(STAFF_FILE))) {
return (ArrayList<HotelStaff>) ois.readObject();
} catch (IOException | ClassNotFoundException e) {
System.err.println("Error loading staff data: " + e.getMessage());
return new ArrayList<>();
}
```

```
}
public void saveToCSV(ArrayList<HotelStaff> staffList, String filename) {
try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
writer.println("Name,Surname,StaffID,DateOfBirth,Role,Additional Info");
for (HotelStaff staff : staffList) {
StringBuilder line = new StringBuilder();
line.append(staff.getName()).append(",");
line.append(staff.getSurname()).append(",");
line.append(staff.getStaffID()).append(",");
line.append(staff.getStringDate()).append(",");
if (staff instanceof Manager) {
Manager manager = (Manager) staff;
line.append("Manager,").append(manager.getLicenseNumber());
} else if (staff instanceof HouseKeeper) {
HouseKeeper houseKeeper = (HouseKeeper) staff;
line.append("HouseKeeper,").append(houseKeeper.getYearsOfExperience());
}
writer.println(line.toString());
}
} catch (IOException e) {
System.err.println("Error saving to CSV: " + e.getMessage());
}
}
}
```

Q16: How would you make the HotelStaff classes serializable?

Implementation:

Java

```
import java.io.Serializable;

public abstract class HotelStaff implements Serializable {

private static final long serialVersionUID = 1L;

// existing fields and methods

// Custom serialization if needed

private void writeObject(ObjectOutputStream oos) throws IOException {

oos.defaultWriteObject();

// Custom serialization logic

}

private void readObject(ObjectInputStream ois) throws IOException,
ClassNotFoundException {

ois.defaultReadObject();

// Custom deserialization logic

}

}

public class Manager extends HotelStaff {

private static final long serialVersionUID = 2L;

// existing implementation

}

public class HouseKeeper extends HotelStaff {

private static final long serialVersionUID = 3L;

// existing implementation

}
```

ADVANCED OOP CONCEPTS

Q17: How would you implement method overloading in the hotel system?

Implementation examples:

```
public class WestminsterHotelManager implements HotelManager {
// Method overloading for adding staff
public void addStaff(String name, String surname, String staffID, LocalDate dob) {
// Generic staff addition
}
public void addStaff(Manager manager) {
addStaffToList(manager);
}
public void addStaff(HouseKeeper houseKeeper) {
addStaffToList(houseKeeper);
}
// Method overloading for searching
public HotelStaff findStaff(String staffID) {
// Search by ID
}
public ArrayList<HotelStaff> findStaff(String name, String surname) {
// Search by name and surname
}
public ArrayList<HotelStaff> findStaff(LocalDate dateOfBirth) {
// Search by date of birth
}
// Method overloading for reports
```

```
public void generateReport() {
// Generate basic report
}
public void generateReport(String reportType) {
// Generate specific type report
}
public void generateReport(LocalDate startDate, LocalDate endDate) {
// Generate report for date range
}
}
```

Q18: Explain constructor chaining in the inheritance hierarchy.

Constructor chaining example:

```
public abstract class HotelStaff {
protected String name;
protected String surname;
// Base constructor
public HotelStaff(String name, String surname) {
this.name = name;
this.surname = surname;
System.out.println("HotelStaff constructor called");
}
// Overloaded constructor
public HotelStaff(String name, String surname, String staffID, LocalDate dob) {
this(name, surname); // Call other constructor in same class
this.staffID = staffID;
```

```
this.dateOfBirth = dob;
}
}
public class Manager extends HotelStaff {
private String licenseNumber;
// Constructor calls parent constructor
public Manager(String name, String surname) {
super(name, surname); // Must be first line
System.out.println("Manager constructor called");
}
// Overloaded constructor
public Manager(String name, String surname, String licenseNumber) {
this(name, surname); // Call other constructor in same class
this.licenseNumber = licenseNumber;
}
}
```

Q19: How would you implement equals() and hashCode() methods properly?

Implementation:

```
public abstract class HotelStaff {
@Override
public boolean equals(Object obj) {
if (this == obj) return true;
if (obj == null || getClass() != obj.getClass()) return false;
HotelStaff that = (HotelStaff) obj;
```

```
return Objects.equals(staffID, that.staffID) &&
Objects.equals(name, that.name) &&
Objects.equals(surname, that.surname) &&
Objects.equals(dateOfBirth, that.dateOfBirth);
}
@Override
public int hashCode() {
return Objects.hash(staffID, name, surname, dateOfBirth);
}
}
public class Manager extends HotelStaff {
@Override
public boolean equals(Object obj) {
if (!super.equals(obj)) return false;
if (!(obj instanceof Manager)) return false;
Manager manager = (Manager) obj;
return Objects.equals(licenseNumber, manager.licenseNumber);
}
@Override
public int hashCode() {
return Objects.hash(super.hashCode(), licenseNumber);
}
}
```

Q20: How would you implement the Comparable interface for sorting staff?

Implementation:

```
public abstract class HotelStaff implements Comparable<HotelStaff> {
@Override
public int compareTo(HotelStaff other) {
// Primary sort: by surname
int result = this.surname.compareTo(other.surname);
if (result != 0) return result;
// Secondary sort: by name
result = this.name.compareTo(other.name);
if (result != 0) return result;
// Tertiary sort: by staff ID
return this.staffID.compareTo(other.staffID);
}
}
// Usage:
Collections.sort(hotelStaffList); // Natural ordering
Collections.sort(hotelStaffList, Collections.reverseOrder()); // Reverse order
// Custom comparators
Comparator<HotelStaff> byDateOfBirth =
Comparator.comparing(HotelStaff::getDateOfBirth);
Comparator<HotelStaff> byNameThenSurname = Comparator
.comparing(HotelStaff::getName)
.thenComparing(HotelStaff::getSurname);
hotelStaffList.sort(byDateOfBirth);
```

GUI AND EVENT HANDLING

Q21: How would you add event handling to the HotelTableGUI?

Implementation:

```
public class HotelTableGUI extends JFrame {
public HotelTableGUI(ArrayList<HotelStaff> list) {
// existing constructor code
// Add button event handler
JButton infoButton = new JButton("Info");
infoButton.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
showSelectedStaffInfo();
}
});
// Add table selection listener
myTable.getSelectionModel().addListSelectionListener(new ListSelectionListener() {
@Override
public void valueChanged(ListSelectionEvent e) {
if (!e.getValueIsAdjusting()) {
int selectedRow = myTable.getSelectedRow();
if (selectedRow >= 0) {
enableInfoButton(true);
}
}
}
```

```
});
// Add double-click listener
myTable.addMouseListener(new MouseAdapter() {
@Override
public void mouseClicked(MouseEvent e) {
if (e.getClickCount() == 2) {
showSelectedStaffInfo();
}
}
});
}
private void showSelectedStaffInfo() {
int selectedRow = myTable.getSelectedRow();
if (selectedRow >= 0) {
int modelRow = myTable.convertRowIndexToModel(selectedRow);
HotelStaff staff = list.get(modelRow);
JOptionPane.showMessageDialog(this,
staff.toString(),
"Staff Information",
JOptionPane.INFORMATION_MESSAGE);
}
}
}
```

Q22: How would you implement a custom table cell renderer?

Implementation:

```
public class StaffTableCellRenderer extends DefaultTableCellRenderer {
@Override
public Component getTableCellRendererComponent(JTable table, Object value,
boolean isSelected, boolean hasFocus, int row, int column) {
Component c = super.getTableCellRendererComponent(table, value, isSelected,
hasFocus, row, column);
// Get the staff object for this row
HotelTableModel model = (HotelTableModel) table.getModel();
int modelRow = table.convertRowIndexToModel(row);
// Color coding based on staff type
if (!isSelected) {
if (model.getStaffAt(modelRow) instanceof Manager) {
c.setBackground(new Color(200, 255, 200)); // Light green for managers
} else if (model.getStaffAt(modelRow) instanceof HouseKeeper) {
c.setBackground(new Color(200, 200, 255)); // Light blue for housekeepers
} else {
c.setBackground(Color.WHITE);
}
}
// Special formatting for date column
if (column == 2 && value instanceof String) {
try {
LocalDate date = LocalDate.parse((String) value,
DateTimeFormatter.ofPattern("dd/MM/yyyy"));
```

```
if (date.isBefore(LocalDate.now().minusYears(60))) {
setFont(getFont().deriveFont(Font.BOLD));
}
} catch (Exception e) {
// Handle parsing error
}
}
return c;
}
}
// Usage in HotelTableGUI:
myTable.setDefaultRenderer(Object.class, new StaffTableCellRenderer());
```

GENERICS AND COLLECTIONS

Q23: How would you implement a generic Repository pattern for the hotel system?

Implementation:

```
public interface Repository<T, ID> {
void save(T entity);
T findById(ID id);
List<T> findAll();
void update(T entity);
void delete(ID id);
```

```
boolean exists(ID id);
}
public class HotelStaffRepository implements Repository<HotelStaff, String> {
private Map<String, HotelStaff> staffStorage = new HashMap<>();
@Override
public void save(HotelStaff staff) {
staffStorage.put(staff.getStaffID(), staff);
}
@Override
public HotelStaff findById(String staffID) {
return staffStorage.get(staffID);
}
@Override
public List<HotelStaff> findAll() {
return new ArrayList<>(staffStorage.values());
}
@Override
public void update(HotelStaff staff) {
if (staffStorage.containsKey(staff.getStaffID())) {
staffStorage.put(staff.getStaffID(), staff);
}
}
@Override
public void delete(String staffID) {
staffStorage.remove(staffID);
}
```

```
@Override
public boolean exists(String staffID) {
return staffStorage.containsKey(staffID);
}
// Additional query methods
public List<HotelStaff> findByRole(Class<? extends HotelStaff> roleClass) {
return staffStorage.values().stream()
.filter(roleClass::isInstance)
.collect(Collectors.toList());
}
}
```

Q24: How would you implement type-safe collections with bounds?

Implementation:

```
public class HotelDepartment<T extends HotelStaff> {
private List<T> departmentStaff;
private String departmentName;
private int maxCapacity;
public HotelDepartment(String departmentName, int maxCapacity) {
this.departmentName = departmentName;
this.maxCapacity = maxCapacity;
this.departmentStaff = new ArrayList<>();
}
public boolean addStaff(T staff) {
```

```
if (departmentStaff.size() < maxCapacity) {
return departmentStaff.add(staff);
}
return false;
}
public List<T> getAllStaff() {
return new ArrayList<>(departmentStaff);
}
public T findStaffById(String staffID) {
return departmentStaff.stream()
.filter(staff -> staff.getStaffID().equals(staffID))
.findFirst()
.orElse(null);
}
}
// Usage:
HotelDepartment<Manager> managementDepartment = new HotelDepartment<>("Management", 5);
HotelDepartment<HouseKeeper> housekeepingDepartment = new HotelDepartment<>("Housekeeping", 20);
Manager manager = new Manager("John", "Doe");
managementDepartment.addStaff(manager); // Type-safe
// This would cause compilation error:
// managementDepartment.addStaff(new HouseKeeper("Jane", "Smith"));
```

Q25: How would you implement wildcard generics for hotel operations?

Implementation:

```
public class HotelOperations {
// Upper bounded wildcard - can read but limited writing
public static double calculateTotalSalary(List<? extends HotelStaff> staffList) {
double total = 0.0;
for (HotelStaff staff : staffList) {
if (staff instanceof Manager) {
total += 5000.0; // Manager salary
} else if (staff instanceof HouseKeeper) {
total += 2500.0; // HouseKeeper salary
}
}
return total;
}
// Lower bounded wildcard - can write but limited reading
public static void addManagers(List<? super Manager> list, Manager... managers) {
for (Manager manager : managers) {
list.add(manager);
}
}
// Unbounded wildcard - very limited operations
public static int getListSize(List<?> list) {
return list.size();
}
```

```
// Generic method with multiple type parameters
public static <T extends HotelStaff, R> List<R> mapStaff(
List<T> staffList,
Function<T, R> mapper) {
return staffList.stream()
.map(mapper)
.collect(Collectors.toList());
}
}
// Usage examples:
List<Manager> managers = Arrays.asList(new Manager("John", "Doe"));
List<HouseKeeper> housekeepers = Arrays.asList(new HouseKeeper("Jane", "Smith"));
List<HotelStaff> allStaff = new ArrayList<>();
// These all work with upper bounded wildcard
double managerSalaries = HotelOperations.calculateTotalSalary(managers);
double housekeeperSalaries = HotelOperations.calculateTotalSalary(housekeepers);
double totalSalaries = HotelOperations.calculateTotalSalary(allStaff);
// Lower bounded wildcard usage
HotelOperations.addManagers(allStaff, new Manager("Bob", "Wilson"));
```

LAMBDA EXPRESSIONS AND STREAMS

Q26: How would you use streams to process hotel staff data?

Implementation:

```
public class HotelAnalytics {
private List<HotelStaff> staffList;
public HotelAnalytics(List<HotelStaff> staffList) {
this.staffList = staffList;
}
// Find all managers with license numbers starting with "MG"
public List<Manager> findManagersByLicensePrefix(String prefix) {
return staffList.stream()
.filter(staff -> staff instanceof Manager)
.map(staff -> (Manager) staff)
.filter(manager -> manager.getLicenseNumber().startsWith(prefix))
.collect(Collectors.toList());
}
// Get average years of experience for housekeepers
public OptionalDouble getAverageHousekeeperExperience() {
return staffList.stream()
.filter(staff -> staff instanceof HouseKeeper)
.mapToInt(staff -> ((HouseKeeper) staff).getYearsOfExperience())
.average();
}
// Group staff by role
public Map<String, List<HotelStaff>> groupStaffByRole() {
```

```
return staffList.stream()
.collect(Collectors.groupingBy(staff -> {
if (staff instanceof Manager) return "Manager";
if (staff instanceof HouseKeeper) return "HouseKeeper";
return "Other";
}));
}
// Find oldest staff member
public Optional<HotelStaff> findOldestStaff() {
return staffList.stream()
.min(Comparator.comparing(HotelStaff::getDateOfBirth));
}
// Get staff names sorted alphabetically
public List<String> getStaffNamesSorted() {
return staffList.stream()
.map(staff -> staff.getName() + " " + staff.getSurname())
.sorted()
.collect(Collectors.toList());
}
// Count staff by decade of birth
public Map<String, Long> countStaffByBirthDecade() {
return staffList.stream()
.collect(Collectors.groupingBy(
staff -> {
int year = staff.getDateOfBirth().getYear();
int decade = (year / 10) * 10;
```

```
return decade + "s";
},
Collectors.counting()
));
}
// Check if any housekeeper has more than 10 years experience
public boolean hasExperiencedHousekeeper() {
return staffList.stream()
.filter(staff -> staff instanceof HouseKeeper)
.anyMatch(staff -> ((HouseKeeper) staff).getYearsOfExperience() > 10);
}
}
```

Q27: How would you implement functional interfaces for hotel operations?

Implementation:

```
// Custom functional interfaces
@FunctionalInterface
public interface StaffValidator {
boolean validate(HotelStaff staff);
}
@FunctionalInterface
public interface StaffProcessor<T> {
T process(HotelStaff staff);
}
```

```
@FunctionalInterface
public interface HotelOperation {
void execute();
}
public class FunctionalHotelOperations {
private List<HotelStaff> staffList;
public FunctionalHotelOperations(List<HotelStaff> staffList) {
this.staffList = staffList;
}
// Using predefined functional interfaces
public List<HotelStaff> filterStaff(Predicate<HotelStaff> condition) {
return staffList.stream()
.filter(condition)
.collect(Collectors.toList());
}
public <R> List<R> transformStaff(Function<HotelStaff, R> transformer) {
return staffList.stream()
.map(transformer)
.collect(Collectors.toList());
}
public void performOnEachStaff(Consumer<HotelStaff> action) {
staffList.forEach(action);
}
// Using custom functional interfaces
public List<HotelStaff> validateAndFilter(StaffValidator validator) {
return staffList.stream()
```

```
.filter(validator::validate)
.collect(Collectors.toList());
}
public <T> List<T> processAllStaff(StaffProcessor<T> processor) {
return staffList.stream()
.map(processor::process)
.collect(Collectors.toList());
}
public void executeOperation(HotelOperation operation) {
operation.execute();
}
}
// Usage examples with lambda expressions:
FunctionalHotelOperations ops = new FunctionalHotelOperations(staffList);
// Filter managers
List<HotelStaff> managers = ops.filterStaff(staff -> staff instanceof Manager);
// Get all staff names
List<String> names = ops.transformStaff(staff -> staff.getName() + " " + staff.getSurname());
// Print all staff info
ops.performOnEachStaff(staff -> System.out.println(staff.toString()));
// Custom validation
List<HotelStaff> validStaff = ops.validateAndFilter(staff -> {
if (staff instanceof Manager) {
return ((Manager) staff).getLicenseNumber() != null;
}
if (staff instanceof HouseKeeper) {
```

```
return ((HouseKeeper) staff).getYearsOfExperience() >= 0;
}
return true;
});
// Method references
List<String> staffIds = ops.transformStaff(HotelStaff::getStaffID);
ops.performOnEachStaff(System.out::println);
```

MULTITHREADING ADVANCED CONCEPTS

Q28: How would you implement thread-safe lazy initialization for hotel resources?

Implementation:

```
public class HotelResourceManager {
// Double-checked locking pattern
private volatile DatabaseConnection dbConnection;
private final Object dbLock = new Object();
public DatabaseConnection getDatabaseConnection() {
if (dbConnection == null) {
synchronized (dbLock) {
if (dbConnection == null) {
dbConnection = new DatabaseConnection();
}
}
}
return dbConnection;
```

```
}
// Initialization-on-demand holder pattern (preferred)
private static class LoggerHolder {
private static final Logger LOGGER =
LoggerFactory.getLogger(HotelResourceManager.class);
}
public static Logger getLogger() {
return LoggerHolder.LOGGER;
}
// Using enum for singleton (thread-safe by default)
public enum HotelConfiguration {
INSTANCE;
private final Properties config;
HotelConfiguration() {
config = loadConfiguration();
}
private Properties loadConfiguration() {
// Load configuration from file
return new Properties();
}
public String getProperty(String key) {
return config.getProperty(key);
}
}
}
```

Q29: How would you implement a thread-safe cache for hotel data?

Implementation:

```
public class HotelDataCache<K, V> {
private final ConcurrentHashMap<K, V> cache;
private final ReadWriteLock lock = new ReentrantReadWriteLock();
private final Lock readLock = lock.readLock();
private final Lock writeLock = lock.writeLock();
private final int maxSize;
private final long timeToLive;
// Entry wrapper with timestamp
private static class CacheEntry<V> {
private final V value;
private final long timestamp;
CacheEntry(V value) {
this.value = value;
this.timestamp = System.currentTimeMillis();
}
boolean isExpired(long ttl) {
return System.currentTimeMillis() - timestamp > ttl;
}
}
private final ConcurrentHashMap<K, CacheEntry<V>> cacheWithTTL;
public HotelDataCache(int maxSize, long timeToLiveMs) {
this.cache = new ConcurrentHashMap<>();
this.cacheWithTTL = new ConcurrentHashMap<>();
```

```
this.maxSize = maxSize;
this.timeToLive = timeToLiveMs;
// Start cleanup thread
startCleanupThread();
}
public V get(K key) {
readLock.lock();
try {
CacheEntry<V> entry = cacheWithTTL.get(key);
if (entry != null && !entry.isExpired(timeToLive)) {
return entry.value;
}
return null;
} finally {
readLock.unlock();
}
}
public void put(K key, V value) {
writeLock.lock();
try {
if (cacheWithTTL.size() >= maxSize) {
evictOldestEntry();
}
cacheWithTTL.put(key, new CacheEntry<>(value));
} finally {
writeLock.unlock();
```

```
}
}
private void evictOldestEntry() {
// Simple LRU eviction - remove first entry
if (!cacheWithTTL.isEmpty()) {
K firstKey = cacheWithTTL.keys().nextElement();
cacheWithTTL.remove(firstKey);
}
}
private void startCleanupThread() {
Thread cleanupThread = new Thread(() -> {
while (!Thread.currentThread().isInterrupted()) {
try {
Thread.sleep(60000); // Clean every minute
cleanExpiredEntries();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
});
cleanupThread.setDaemon(true);
cleanupThread.start();
}
private void cleanExpiredEntries() {
writeLock.lock();
```

```
try {
cacheWithTTL.entrySet().removeIf(entry -> entry.getValue().isExpired(timeToLive));
} finally {
writeLock.unlock();
}
}
}
// Usage: HotelDataCache<String, HotelStaff> staffCache = new HotelDataCache<>(100, 300000); // 5 minutes TTL
// staffCache.put("EMP001", manager);
// HotelStaff cachedStaff = staffCache.get("EMP001");
```

Q30: How would you implement a work-stealing queue for hotel tasks?

Implementation:

```
public class HotelTaskProcessor {
private final ForkJoinPool forkJoinPool;
public HotelTaskProcessor() {
this.forkJoinPool = new ForkJoinPool();
}
// Abstract task class
public abstract static class HotelTask extends RecursiveTask<String> {
protected abstract String executeTask();
@Override
protected String compute() {
```

```
return executeTask();
}
}
// Concrete task implementations
public static class RoomCleaningTask extends HotelTask {
private final int roomNumber;
private final HouseKeeper houseKeeper;
public RoomCleaningTask(int roomNumber, HouseKeeper houseKeeper) {
this.roomNumber = roomNumber;
this.houseKeeper = houseKeeper;
}
@Override
protected String executeTask() {
// Simulate room cleaning
try {
Thread.sleep(2000 + (int)(Math.random() * 3000)); // 2-5 seconds
return "Room " + roomNumber + " cleaned by " + houseKeeper.getName();
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
return "Room " + roomNumber + " cleaning interrupted";
}
}
}
public static class MaintenanceTask extends HotelTask {
private final String taskDescription;
private final String assignedTo;
```

```
public MaintenanceTask(String taskDescription, String assignedTo) {

this.taskDescription = taskDescription;

this.assignedTo = assignedTo;

}

@Override

protected String executeTask() {

try {

Thread.sleep(1000 + (int)(Math.random() * 2000)); // 1-3 seconds

return "Maintenance: " + taskDescription + " completed by " + assignedTo;

} catch (InterruptedException e) {

Thread.currentThread().interrupt();

return "Maintenance:...
```

I have provided the questions and answers from the file. However, I am unable to convert the content into a PDF format or apply specific formatting beyond what is present in the original markdown. My current capabilities do not include file format conversion or advanced document rendering.